Disease Classification from Symptom Text: Frozen BERT vs. TF-IDF + Logistic Regression

NLP Group Project Report. Krish Mehta, Austin Badal, Muaaz Shaikh, Adnan Bagdadi. October 2026.

1. Abstract

People describe their symptoms in everyday language, and a useful assistant has to map that free text to the most probable disease. This project builds and compares two NLP classifiers on the Symptom2Disease dataset, which contains short symptom descriptions labelled with one of 24 diseases. Model A uses a pretrained BERT encoder as a frozen feature extractor, with a small dense classification head trained on top of its [CLS] embedding. Model B is a traditional baseline: TF-IDF features (unigrams and bigrams) with Logistic Regression. Both models use identical text cleaning and the same stratified 70/15/15 train/validation/test split, so the comparison is fair.

On the held-out test set of 173 descriptions, the BERT model reached 95.38% accuracy (Macro-F1 0.9543, Weighted-F1 0.9532) and TF-IDF + Logistic Regression reached 94.80% accuracy (Macro-F1 0.9486, Weighted-F1 0.9483). The gap is a single test sample, so the two approaches are effectively tied on this dataset, while TF-IDF is far cheaper to train and easier to interpret. The work also includes a prediction demo, saved model weights and a Gradio web interface that runs both models side by side. This is an academic prototype and not a medical diagnosis tool.

2. Introduction and Motivation

Self-reported symptoms are rarely written in clinical vocabulary. One person writes "my skin is peeling around my knees and elbows", another writes "there is a silver-like dusting on my lower back", and both may describe the same condition. Automatic systems that read such text could support triage tools, symptom checkers and health chatbots, but only if they handle varied wording reliably.

Modern language models such as BERT capture context and meaning beyond exact word overlap, which makes them an obvious candidate. However, they are expensive to run and harder to explain. Classic sparse methods such as TF-IDF with a linear classifier are cheap and transparent, and are often strong when texts are short and keyword-rich. This project tests which approach is the better fit for symptom text, rather than assuming that the more complex model wins.

3. Problem Statement

Symptom descriptions are written in varied wording and lengths (12 to 55 words in this dataset). The task is to map each description to exactly one of 24 diseases. The open question is whether contextual embeddings from a pretrained BERT model outperform a simple TF-IDF baseline on short symptom text, and what each approach costs in training effort and interpretability.

4. Objectives

- Classify diseases from natural-language symptom descriptions using two distinct NLP approaches.
- Compare both approaches on the exact same data split using Accuracy, Macro-F1 and Weighted-F1, plus per-class reports and confusion matrices.
- Analyse where each model succeeds and fails, and what the results imply for model choice.
- Provide custom prediction for new text, persist the trained model, and offer a simple interface to try both models.

5. Scope

The project covers five areas: (1) data preparation, meaning loading, inspecting, cleaning and preprocessing the text; (2) exploratory analysis with class distribution, description length and word clouds; (3) model implementation for both approaches on a stratified 70/15/15 split; (4) evaluation and comparison; and (5) inference, which includes custom symptom prediction, model saving and a web demo. Out of scope are end-to-end fine-tuning of BERT, clinical validation and any real patient data.

6. Dataset

The project uses the Symptom2Disease dataset: a table with a free-text symptom description (`text`) and a disease name (`label`). The raw file has 1,200 rows and an extra index column (`Unnamed: 0`), with no missing values.

Property	Value
Raw rows	1,200
Rows after removing duplicates	1,153 (47 duplicates removed)
Features	text, label

Property	Value
Disease classes	24
Description length	mean 30.6 words (std 6.7), median 30, range 12 to 55, middle half 26 to 35
Class balance	roughly 38 to 50 samples per class after de-duplication

An important detail: the first duplicate check reported zero duplicates because the index column made every row unique. Only after dropping `Unnamed: 0` did the 47 repeated descriptions appear. Removing them matters because exact duplicates split across train and test would inflate the test score.

The 24 classes are Acne, Arthritis, Bronchial Asthma, Cervical spondylosis, Chicken pox, Common Cold, Dengue, Dimorphic Hemorrhoids, Fungal infection, Hypertension, Impetigo, Jaundice, Malaria, Migraine, Pneumonia, Psoriasis, Typhoid, Varicose Veins, allergy, diabetes, drug reaction, gastroesophageal reflux disease, peptic ulcer disease and urinary tract infection.

6.1 Exploratory analysis

The word cloud shows the dominant terms are skin, feel, headache, pain, fever, throat, rash, cough and joint. Description length is roughly bell-shaped and peaks at 26 to 36 words. Because the classes are nearly balanced, accuracy and Macro-F1 are directly comparable and no resampling was needed.

6.2 Train, validation and test split

The data is divided in two stratified steps with `random_state=101`: first 70% for training and 30% held back, then that 30% is split 50/50 into validation and test. Stratification keeps the class proportions the same in all three sets. Both models use exactly these sets.

Set	Samples	Purpose
Training	807	Fit the models
Validation	173	Early stopping and checkpoint selection
Test	173	Final, held-out comparison

Each class has only 6 to 8 samples in the test set, so a single misclassification moves a per-class score noticeably.

7. Methodology

7.1 Overall pipeline

The workflow is: load and inspect the data, drop the index column and duplicates, clean the text, encode the labels, create the stratified split, then train and evaluate Model A and Model B on identical inputs. Results are compared on the test set and the models are exposed through a prediction function and a web interface.

7.2 Text preprocessing

Every description passes through the same cleaning functions, applied in this order: remove HTML, remove email addresses, remove underscores, remove digits, remove links, remove special characters, remove stop words, strip non-ASCII characters, remove punctuation and lowercase. The helper functions come from a small helper module that the notebook downloads at run time. The same pipeline is applied to new user input at prediction time, so training and inference see the same kind of text.

Example. Original: "I have been experiencing a skin rash on my arms, legs, and torso for the past few weeks. It is red, itchy, and covered in dry, scaly patches." Cleaned: "experiencing skin rash arms legs torso past weeks red itchy covered dry scaly patches", which reduces 28 words to 14 content words.

7.3 Label encoding

The 24 disease names are converted to integers 0 to 23 with scikit-learn's `LabelEncoder`. The mapping is saved to `disease_label_mapping.json` so predictions can be turned back into names.

8. Model A: Frozen BERT with a Dense Classification Head

8.1 Encoder

The encoder is the pretrained Hugging Face checkpoint `ajtamayoh/Symptoms_to_Diagnosis_SonatafyAI_BERT_v1`, loaded with PyTorch `AutoModel`. Its original classifier weights are ignored (the loader reports them as unexpected keys), and only the transformer encoder is used. Text is tokenized with a maximum length of 170 tokens. The representation of the first token, [CLS], a 768-dimensional vector, is taken as the sentence embedding.

The encoder is frozen: none of its weights are updated. In the Keras model it is wrapped in a custom layer (`TFBertBridge`) that runs the PyTorch forward pass inside `tf.numpy_function` and returns no gradients. This is why the Keras summary shows zero

parameters for the bridge layer; the BERT weights are held outside Keras's parameter count.

8.2 Classification head

Layer	Output size	Notes
[CLS] embedding	768	From frozen BERT
Dense + ReLU	128	Batch normalization, dropout 0.2
Dense + ReLU	64	Batch normalization, dropout 0.1
Dense + ReLU	32	None
Dense + softmax	24	One probability per disease

The head has 110,328 parameters in total: 109,944 trainable and 384 non-trainable (batch normalization statistics).

8.3 Training procedure

Because the encoder never changes, the notebook computes the [CLS] embeddings once for the training and validation sets (on a GPU) and trains the head directly on those 768-dimensional vectors. This makes training very fast: after the first epoch, which includes start-up cost, each epoch takes well under a second. The trained head weights are then copied layer by layer into the full BERT model, so predictions can still be made from raw tokenized text.

Setting	Value
Optimizer	Adam, learning rate 1e-3, gradient clipping (clipnorm 1.0)
Loss	Sparse categorical cross-entropy
Batch size	16
Maximum epochs	40
Early stopping	Monitor validation accuracy, patience 10, restore best weights
Checkpointing	Save best-validation-accuracy head weights (`symptoms_disease_classifier_weights.weights.h5`)

Two callbacks that the notebook defines (a time-based learning-rate decay schedule and a full-model checkpoint) are not passed to the final training call, so they had no effect on the reported results.

9. Model B: TF-IDF + Logistic Regression

The baseline turns each cleaned description into a sparse vector of weighted word and phrase features and classifies it with a linear model.

Component	Setting
Vectorizer	TF-IDF with unigrams and bigrams (`ngram_range=(1, 2)`)
Feature cap and filter	`max_features=50000`, `min_df=2` (resulting in 3,063 features)
Term scaling	Sublinear TF (`sublinear_tf=True`)
Classifier	Logistic Regression, `max_iter=2000`, `class_weight='balanced'`, `random_state=101`

The vectorizer is fitted on the training text only and then used to transform the validation and test text, which prevents information from the held-out sets leaking into the features. Training takes seconds on a CPU.

10. Experimental Setup

The notebook runs on Google Colab with TensorFlow 2.21.0 (Keras), PyTorch, Hugging Face `transformers`, scikit-learn, pandas, seaborn, matplotlib and WordCloud. A GPU is used only to compute the BERT embeddings. Models are evaluated on the 173-sample test set using Accuracy, Macro-F1 (each disease weighted equally), Weighted-F1 (weighted by class size), per-class precision, recall and F1, and confusion matrices. The Gradio interface is added with a separate package install.

11. Results

11.1 BERT head training behaviour

Validation accuracy rose quickly: 72.3% after epoch 1, 90.2% after epoch 3 and 97.1% by epoch 6. After that it fluctuated between roughly 94.8% and 98.8% while training accuracy approached 99.7%. The best validation accuracy was 98.84% at epoch 23

(validation loss 0.0679). Validation accuracy did not improve for the next 10 epochs, so early stopping triggered at epoch 33 and the weights from epoch 23 were restored.

Two points follow from this. First, the head fits the training data almost perfectly, and the gap between training and validation loss shows mild overfitting, which dropout and batch normalization only partly control. Second, because the best epoch was chosen on the validation set, the 98.84% validation figure is optimistic. The unbiased estimate is the test score below.

11.2 Test-set comparison

Metric	BERT + dense head	TF-IDF + Logistic Regression
Accuracy	95.38%	94.80%
Macro-F1	0.9543	0.9486
Weighted-F1	0.9532	0.9483
Correct predictions (of 173)	165	164
Test loss (cross-entropy)	0.2895	Not reported

For reference, TF-IDF + Logistic Regression scored 95.38% accuracy and 0.9544 Macro-F1 on the validation set, so its test result is about half a point lower, as expected for a final held-out estimate.

BERT is ahead by 0.58 percentage points, which equals one additional correct sample out of 173. With a test set this small, that difference is within noise, and the fair conclusion is that the two models perform equally well on this dataset.

11.3 Per-class performance

Twelve of the 24 classes (Acne, Arthritis, Cervical spondylosis, Chicken pox, Common Cold, Dimorphic Hemorrhoids, Fungal infection, Jaundice, Malaria, Migraine, Pneumonia and allergy) scored an F1 of 1.00 with both models. The table lists the other twelve, where at least one model fell short.

Class	BERT F1	TF-IDF + LR F1
Bronchial Asthma	0.92	1.00
Dengue	0.93	0.89
Hypertension	0.94	1.00

Class	BERT F1	TF-IDF + LR F1
Impetigo	0.89	0.94
Psoriasis	0.92	0.77
Typhoid	0.93	0.93
Varicose Veins	0.77	0.92
diabetes	0.83	0.86
drug reaction	0.94	0.93
gastroesophageal reflux disease	1.00	0.77
peptic ulcer disease	0.88	0.75
urinary tract infection	0.94	1.00

The two models make different mistakes. BERT is weakest on Varicose Veins (F1 0.77), diabetes (recall 0.71) and peptic ulcer disease (0.88), while it classifies gastroesophageal reflux disease perfectly. TF-IDF is weakest on peptic ulcer disease (0.75, precision 0.67), Psoriasis (recall 0.71) and gastroesophageal reflux disease (0.77), while it is perfect on Bronchial Asthma, Hypertension and urinary tract infection. Peptic ulcer disease is the one class both models find hard. For TF-IDF, the low recall for reflux disease together with the low precision for peptic ulcer is consistent with these two digestive conditions being confused, since they share symptoms such as stomach pain and acidity. The exact confusion pairs are in the confusion matrices produced by the notebook.

11.4 Custom symptom prediction

The BERT model was tested on a new description: "I've been experiencing stiffness in my neck and my joints have swollen. I've also been experiencing pain when I walk." After cleaning ("i ve experiencing stiffness neck joints swollen i ve also experiencing pain walk") it predicted Arthritis with 83.78% confidence. The top five predictions were:

Rank	Disease	Probability
1	Arthritis	83.78%
2	Migraine	13.25%
3	Common Cold	0.93%
4	allergy	0.89%
5	Dimorphic Hemorrhoids	0.45%

The TF-IDF model, given a similar sentence ("I have stiffness in my neck, swollen joints, and pain when walking."), also predicted Arthritis, but with a probability of only 20.66%. Both models agree on the answer; they differ in confidence because a softmax over BERT features is sharper than a regularized linear model over short sparse vectors. Probabilities from the two models should therefore be compared by rank, not by value.

12. Web Interface (Gradio)

To make the comparison easy to demonstrate, the notebook builds a Gradio web app that runs both trained models side by side. The user types a symptom description and receives the top five predictions from the BERT model and from TF-IDF + Logistic Regression, plus a one-line summary stating whether the two models agree or disagree. The app cleans the input with the same pipeline used in training, handles empty input and input that is empty after cleaning, and shows an "academic project only, not medical advice" notice. Four example descriptions (skin rash, joint stiffness, dengue-like fever, urinary burning) are provided. When launched with sharing enabled, Gradio creates a temporary public link that lasts up to a week; permanent hosting would need a deployment such as Hugging Face Spaces.

13. Saved Artifacts

File	Contents
`symptoms_disease_classifier_weights.weights.h5`	Best-validation-accuracy weights of the classification head, written during training
`symptoms_disease_classifier_final_weights.weights.h5`	Weights of the full BERT model after the trained head was copied in, saved at the end

File	Contents
`disease_label_mapping.json`	Mapping from class index (0 to 23) to disease name

Only weights are saved, not the whole model, because the custom PyTorch bridge layer does not serialize cleanly with Keras. To reuse the model, rebuild the same architecture, load the BERT checkpoint from Hugging Face and then load the weights file.

14. Discussion

The two models are tied. BERT's lead of one sample is not meaningful with 173 test samples and a single split. Anyone choosing between them should therefore decide on cost, speed and explainability rather than on accuracy.

Why a simple model does so well. The descriptions are short (about 30 words), keyword-rich and fairly templated, and each disease has distinctive vocabulary such as "blackheads" for acne or "nails" and "scales" for psoriasis. When the signal sits in specific words, TF-IDF with bigrams captures it almost as well as a contextual encoder. BERT's strength, understanding meaning beyond exact word overlap, has little room to help on text like this.

Cost and practicality. TF-IDF + Logistic Regression trains in seconds on a CPU and its decisions can be read from the term weights. The BERT route needs a large pretrained model, a GPU to embed the data efficiently (a CPU pass through the PyTorch bridge took about 52 to 55 seconds for the 173 test samples), and a custom bridge to combine PyTorch and Keras. Training only the head is cheap: the first epoch took about 31 seconds including start-up and later epochs well under a second. The model is also a black box by comparison.

Complementary errors. Because the models fail on different classes, combining them is a promising idea, for example by averaging their probabilities. The notebook's web app already shows both outputs together and flags disagreement, which is a simple practical use of this.

Validation versus test. BERT scored 98.84% on validation but 95.38% on test, a gap of about 3.5 points. This is the expected optimism from picking the best epoch on the validation set, and it is why the test set, used once at the end, is the number to trust.

Expected outcomes versus achieved

Expected outcome	Status	Evidence
Accurate multi-class prediction	Achieved	95.38% (BERT) and 94.80% (TF-IDF + LR) test accuracy
BERT vs. TF-IDF comparison	Achieved	Accuracy, Macro-F1, Weighted-F1, per-class reports and confusion matrices
Custom symptom prediction	Achieved	Example predicted as Arthritis by both models (BERT 83.78%)
Saved model weights	Achieved	Two `.weights.h5` files and the label mapping JSON
Interface to try both models	Added	Gradio app with side-by-side top-5 output

15. Limitations

- **Small data and one split.** There are 1,153 descriptions and 173 test samples, evaluated on one random split. There is no variance estimate or significance test, so differences of a point or two are noise.
- **Templated text.** The descriptions are short and fairly uniform. Real patient text is longer, noisier and more ambiguous, so these scores are probably optimistic for real use.
- **Preprocessing order.** Stop-word removal runs before lowercasing, so capitalized stop words such as "My", "The" and "There" survive (visible in the cleaned examples, such as "my skin peeling"). Contractions are also split ("I've" becomes "i ve"). The effect is small but easy to fix.
- **Unverified encoder history.** The BERT checkpoint was already trained for symptom-to-diagnosis tasks. We did not verify which data it saw, so overlap with this dataset cannot be ruled out, and any overlap could favour BERT.
- **Unused training options.** A learning-rate decay schedule and a full-model checkpoint are defined but not used in the final training call.
- **Closed set of 24 diseases.** The model always returns one of the 24 diseases and has no way to say "none of these" or "I am not sure".
- **Environment dependence.** The notebook is written for Google Colab and downloads a helper file from an external repository at run time, which can break reproducibility if that file changes.

- **Not a medical device.** The system has not been clinically validated and must not be used for diagnosis.

16. Future Work

- Fine-tune BERT end to end and train longer, and try clinical language models such as BioBERT.
- Use cross-validation or repeated splits with confidence intervals, and a paired test such as McNemar's test, to decide whether small differences are real.
- Evaluate on longer, noisier, real patient text.
- Lowercase before stop-word removal and test lighter cleaning for BERT, which often works well on near-raw text.
- Build an ensemble of the two models, given their complementary errors.
- Add calibrated confidence and an abstain option that suggests seeing a doctor when the model is unsure.
- Deploy the interface on permanent hosting.

17. Conclusion

Two NLP approaches were built for classifying 24 diseases from symptom text, using identical cleaning and an identical stratified 70/15/15 split. A frozen BERT encoder with a trained dense head reached 95.38% test accuracy (Macro-F1 0.9543), and TF-IDF + Logistic Regression reached 94.80% (Macro-F1 0.9486). The difference is one test sample, so on this small and fairly templated dataset a cheap, transparent linear model matches a deep encoder. The two models make different errors, which suggests that combining them, or testing on harder real-world text, is the most useful next step. All planned deliverables were completed: preprocessing, exploratory analysis, both models, evaluation, custom prediction, saved weights and a side-by-side web interface. The system is a research prototype and not a tool for medical diagnosis.

18. How to Reproduce

1. Open the notebook in Google Colab and select a GPU runtime (needed only to speed up the BERT embeddings).
2. Run all cells from top to bottom so that both models are trained before the comparison cell.
3. When prompted, upload `Symptom2Disease.csv`.
4. Review the comparison table, classification reports and confusion matrices. Results can vary slightly between runs, and the data split is fixed by `random_state=101`.

5. Optionally run the final cells to download the saved weights and label mapping and to launch the Gradio app.

References

- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT.
- Salton, G., and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5).
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.
- Symptom2Disease dataset (Kaggle): free-text symptom descriptions labelled with 24 diseases.
- Hugging Face model checkpoint: ajtamayoh/Symptoms_to_Diagnosis_SonatafyAI_BERT_v1.